Athens University Of Economics and Business

Department of Informatics

COURSE: **PARALLEL PROGRAMMING**

Spring Semester 2025-26

ASSIGNMENT TITLE

Experimenting Different Methods for Workload Distribution of pthread Threads in C++

Name:	**MARAKIS MICHAIL**
Year:	**3rd**
Email:	**p3230267@aueb.gr**
Prof:	**VASILAKIS ANDREAS-ALEXANDROS**

ΠΙΝΑΚΑΣ ΠΕΡΙΕΧΟΜΕΝΩΝ

1 Introduction

The first assignment in this Parallel Programming course focuses on the use of pthread threads in C++ and more specifically the implementation of three methods for workload distribution in an integral computation application. These methods are: **static partitioning** (with and without locks), **cyclic scheduling**, and **dynamic scheduling** using a task queue are examined. The goal is to evaluate their performance and determine the most efficient approach. The integral which these methods are going to be tested is the Trapezoidal Rule for numerical integration:

$$\int_a^b f(x)dx \approx \frac{h}{2}[f(a) + f(b) + 2\sum_{i=1}^{n-1} f(a + ih)],\ h = (b - a)/n$$

1.1 Setup

Metrics were conducted on the following system:

Computer	Processor	Cores	Compiler	O.S.
DESKTOP-MU9QIDD	**Intel i5 vPro 8th Gen**	**4**	**gcc v15.2.0**	**Windows 11**

1.2 Compile and run

First find the name of the file(file_name) where the code is and run this command to compile:

g++ -std=c++17 .<file_name> -o .\<file_name> -pthread

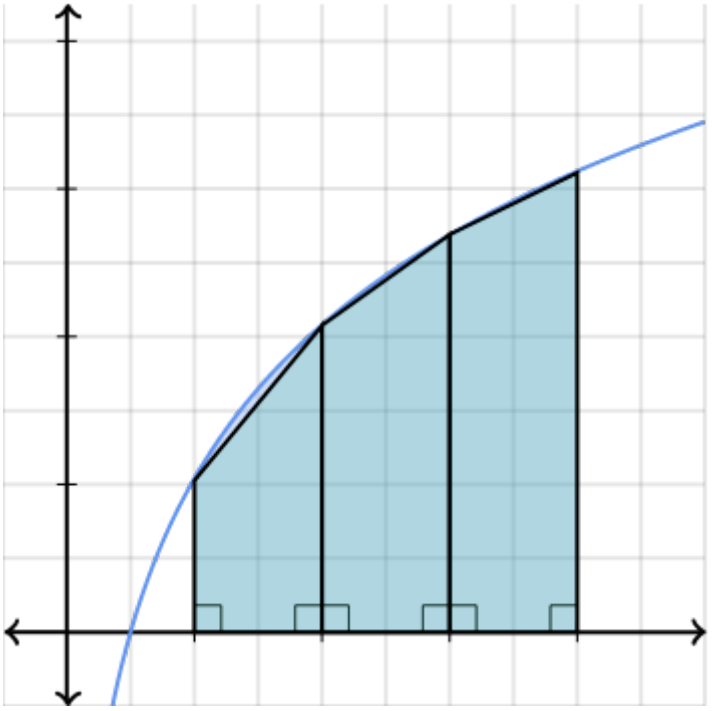
After a successful compilation an .exe file has been created. To run it use the command:

.\<file_name>.exe

1.3 The Problem

The problem is based on approximating an integral using the Riemann sum approach, specifically through the Trapezoidal Rule. According to the definition of the Riemann integral, the continuous space [a,b] is divided into many small subspaces of width $h = \frac{b-a}{N}$, and the integral is approximated as the sum of the function values over these subspaces. As the number of subspaces increases($N \to \infty$) the approximation becomes more accurate and converges to the exact

value of the integral. In the Trapezoidal Rule, the area is approximated using trapezoids instead of rectangles, providing better accuracy. This formulation makes the problem very suitable for parallelization, since each part of the sum is independent and can be computed by different threads, theoretically allowing faster execution compared to the sequential approach.

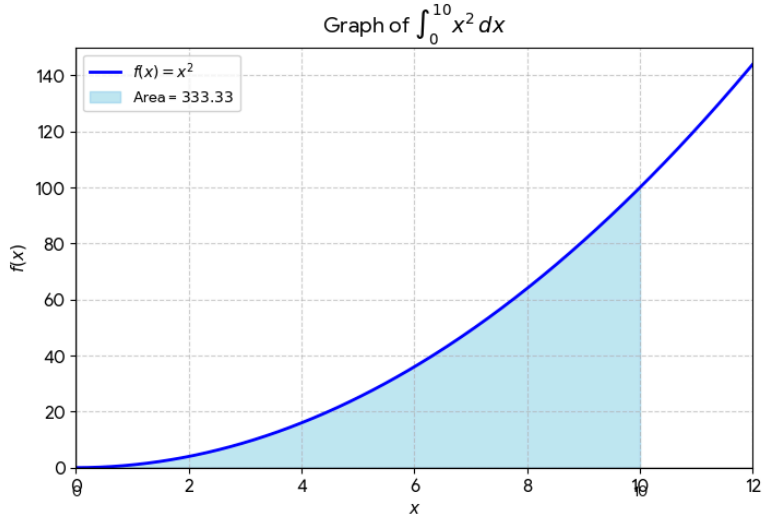


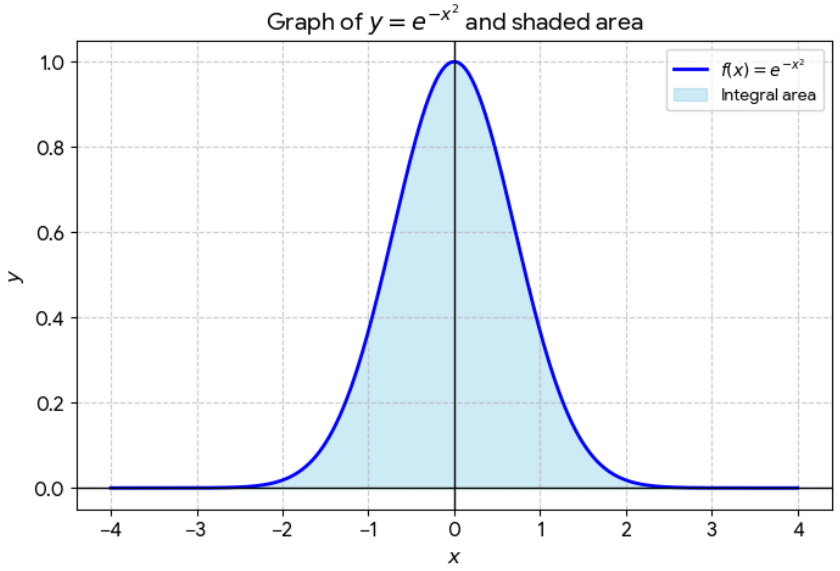
(Application of the Trapezoid Riemann Integral)

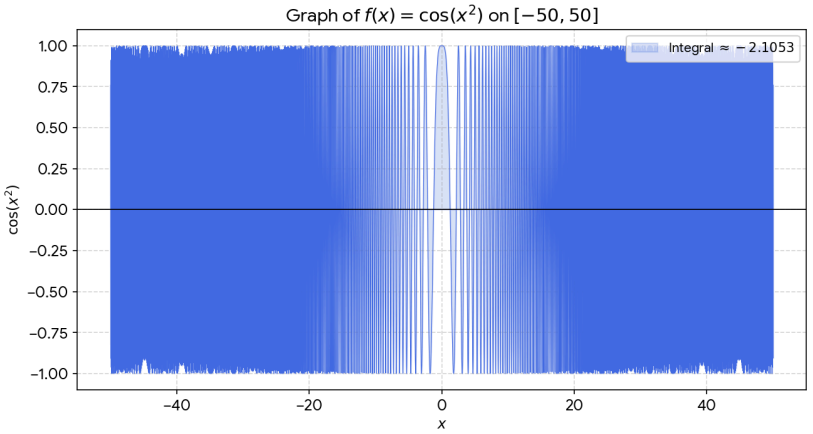
1.4 Integrals to compute

The functions i chose for this assignment are e^{-x^2}, $cos(x^2)$ and x^2. To begin with, the function x^2 at [0.0, 10.0] is simple and computationally lightweight. As we can see from the graph, it is very easy to understand how the Trapezoid Rieman integral works and how the threads will be used in order to compute it. The Gaussian Function, e^{-x^2}, introduces computational cost due to the exponential operation while also providing a well-defined and symmetric shape over the interval [-10.0, 10.0] due to it being symmetrical around the y-axis. This makes it useful for analyzing numerical integration in general. Lastly $cos(x^2)$ at [-50.0, 50.0] is more computationally demanding because of the trigonometric calculation while also exhibiting more complex behavior, making it a good test case for evaluating performance under more challenging conditions. Because of its complex behaviour, an extra experiment will be demonstrated on this function. This extra experiment will be on changing the N(number of subspaces) and we will see how the time of

execution changes, the accuracy of the integral and also how the threads are behaving with different N. Taking all the above into consideration, these functions differ in terms of computational complexity, smoothness, and graphical behavior making the evaluation of performance of workload distribution methods using threads more comprehensive and interesting. Below, the graphs of the functions that will be used can be seen visually.







2 Sequential computation

The sequential implementation is the simplest approach to compute the integral and serves as a baseline for comparison with the parallel methods. In this case, the computation is performed using a single thread, without any form of workload distribution. This simple and basic method allows us to measure the execution time when no parallelism is applied and helps to understand better the performance improvements achieved by the multithreaded implementations that will be tested as well.

2.1 Experiment Results

Results are calculated in seconds

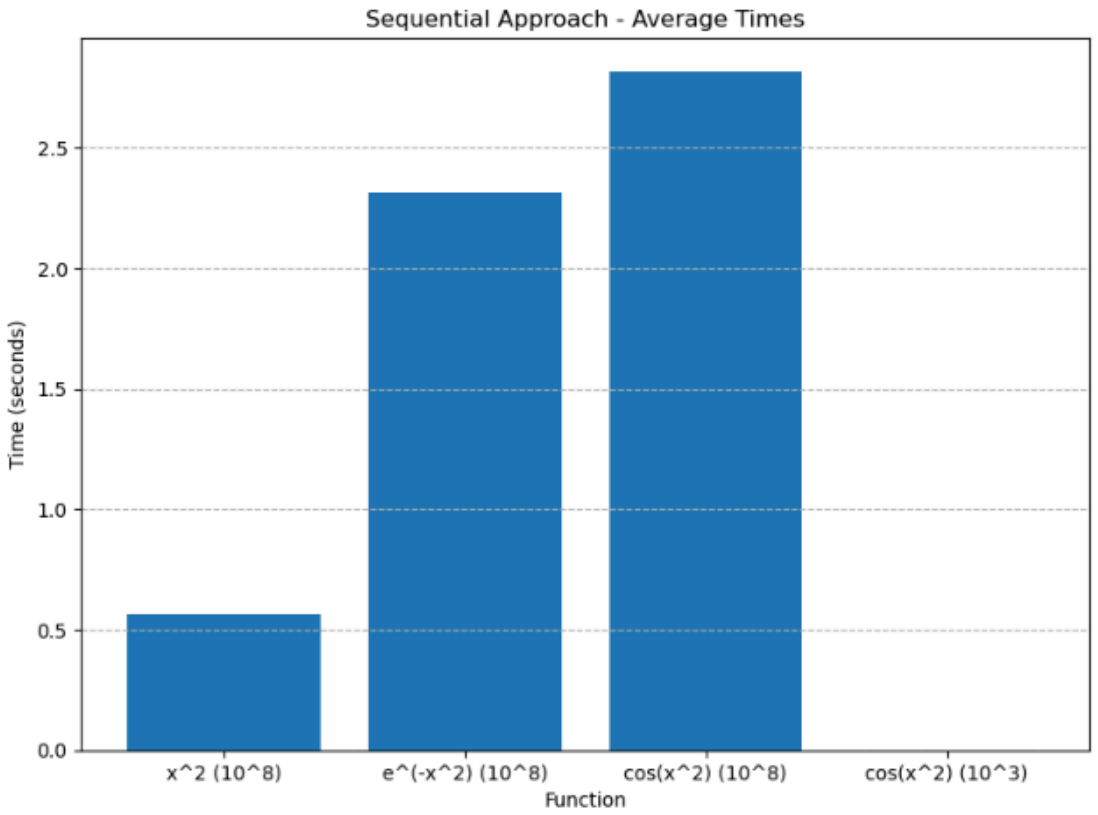
$Func(x)$	first run	second run	third run	fourth run	fifth run	sixth run	avg	res Integr.
x^2 $N = 10^8$	0.55318	0.54775	0.556373	0.544073	0.54690	0.642297	0.565095	333.333
e^{-x^2} $N = 10^8$	2.32375	2.35425	2.30231	2.34819	2.2689	2.29694	2.31572	2.75589
$cos(x^2)$ $N = 10^8$	3.07597	2.72229	2.82977	2.72572	2.79824	2.73848	2.81508	1.24031

$cos(x^2)$ $N = 10^3$	3.64e-05	3.76e-05	3.71e-05	3.77e-05	3.67e-05	6.64e-05	4.19833e-05	4.67453

2.2 Experiment Observations

According to the sequential experiment results, we observe clear differences in computation cost between the functions. The function $f(x) = x^2$. is the fastest, as it is computationally simple, while $f(x) = e^{-x^2}$ requires more time due to the exponential calculation. The most time-consuming case is $f(x) = cos(x^2)$, which involves more complex trigonometric operations. Additionally, when comparing the two cases of $cos(x^2)$, the execution time for N = 10^8 is significantly higher than for N = 10^3, since the number of computations is much larger. However, the result for the smaller N is less accurate, showing the trade-off between execution time and precision.

The sequential approach uses only one thread, meaning all computations are performed one after another. By introducing multithreading, the total workload can be divided among multiple threads, allowing different parts of the integral to be computed in parallel. As a result, the execution time is expected to decrease in the workload distribution methods that are going to be tested, especially for large values of N where the computational cost is high. This improvement is also expected to be more noticeable for complex functions such as $cos(x^2)\ and\ e^{-x^2}$, where each computation is more expensive, making parallel execution more beneficial.



3 Static Partitioning

Static Partitioning is a workload distribution method where the total number of iterations is divided into bordering continual blocks and each thread is assigned a specific portion of the work. Each thread processes its own range of iterations independently, which reduces scheduling overhead. The problem here is when shared variables are used. In this scenario, synchronization mechanisms are required to ensure correct results. In our case, we need to use these types of mechanisms to ensure that every thread is updating the shared variable sum safely.

3.2 Synchronization Mechanisms

3.2.1 Locks

To compute the integral, a shared variable sum is used, which is updated by multiple threads running in parallel. Each thread first calculates its own partial result (local_sum) for a specific portion of the interval,which is determined by its id. The total number of points N is divided equally among the threads, with the last thread handling any remaining elements, so each thread works on the range from

`start = id * piece`, to
`end = (id + 1) * piece` (or `end = N` for the last thread).

```
int piece = N / NUM_THREADS;

int start = id * piece;
int end;

if (id == NUM_THREADS - 1) {
    end = N;
}
else {
    end = (id + 1) * piece;
}
```

For every point in this range, the thread evaluates the function and updates its local_sum: local_sum += 2 * f(x). Once the computation is complete, each thread needs to add its local_sum to the global sum, which is a critical section since sum is shared. To avoid race conditions, a mutex is used. Firstly the thread locks it with pthread_mutex_lock(&mutex), then it updates the shared variable: sum += local_sum, and then unlocks it using pthread_mutex_unlock(&mutex). After all threads have finished, the main thread adds the values of f(a) and f(b) to sum and multiplies the result by h / 2.0 to produce the final value of the integral.This method is known as mutual exclusion and its application ensures that only one thread at a time modifies the shared variable, guaranteeing a correct result.

3.2.1.1 Experiment Results

Results are calculated in seconds

Computing the integral of $f(x) = x^2$ *at* $[a: 0.0, b: 10.0]$
N = `100000000`
Result= 333.333

Threads	first run	second run	third run	fourth run	fifth run	average
1	0.299285	0.297839	0.297973	0.299395	0.29802	0.298502
2	0.157651	0.170163	0.155183	0.166505	0.166485	0.163197
4	0.118714	0.105472	0.0942422	0.106693	0.102216	0.105467
6	0.0863407	0.08093	0.0823605	0.082492	0.0879703	0.0840187
8	0.088351	0.0742829	0.0793014	0.0736732	0.0727708	0.0776759

Computing the integral of $f(x) = e^{-x^2} (Gaussian\ Function)\ at\ [a: -10.0,\ b: 10.0]$

N = 100000000

Result= 1.77245

Threads	first run	second run	third run	fourth run	fifth run	average
1	1.28967	1.28506	1.27326	1.28295	1.27889	1.28197
2	0.649295	0.664589	0.651446	0.657487	0.643087	0.653181
4	0.382912	0.385397	0.37763	0.398706	0.38602	0.386133
6	0.334437	0.308803	0.32362	0.320529	0.316383	0.320754
8	0.26864	0.273246	0.265049	0.266621	0.264809	0.267673

Computing the integral of $f(x) = cos(x^2)\ at\ [a: -50.0,\ b: 50.0]$

N = 100000000

Result= 1.24031

Threads	first run	second run	third run	fourth run	fifth run	average
1	1.62902	1.56954	1.57106	1.57909	1.57746	1.58523
2	0.822496	0.812229	0.835252	0.812997	0.797349	0.816065
4	0.495441	0.464191	0.466597	0.460887	0.455974	0.468618
6	0.418633	0.37376	0.385068	0.37791	0.390246	0.389123
8	0.330229	0.322339	0.326033	0.319493	0.317747	0.323168

Computing the integral of $f(x) = cos(x^2)\ at\ [a: -50.0,\ b: 50.0]$

N = 1000

Result = 4.75051

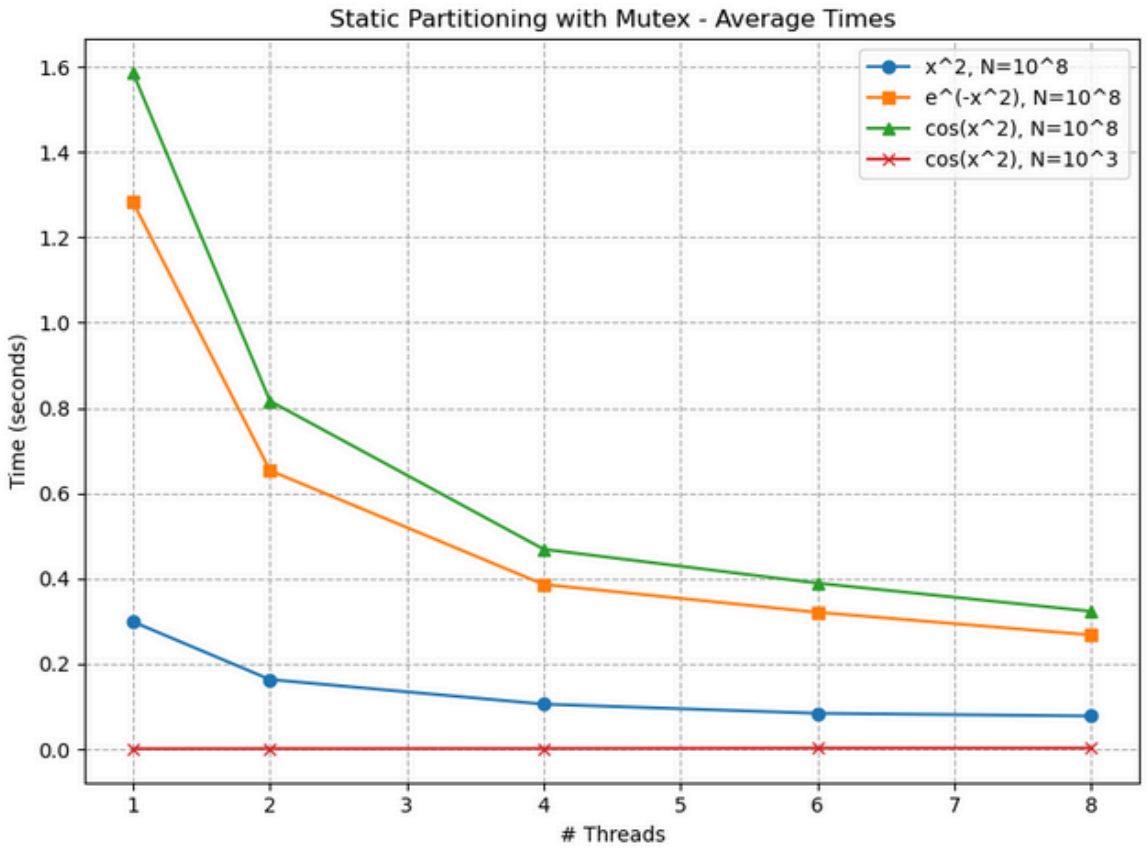
Threads	first run	second run	third run	fourth run	fifth run	average
1	0.0009667	0.0009931	0.0006409	0.001325	0.0014085	0.0010668
2	0.0012348	0.0010248	0.001239	0.0011842	0.0018434	0.0013052
4	0.0016337	0.0015691	0.00133	0.001202	0.0022943	0.0016058

6	0.0031691	0.0018499	0.001886	0.0034541	0.0019222	0.0024562
8	0.0027021	0.0027211	0.002684	0.0022421	0.0022652	0.0025229

Static Partitioning Average Experiment Time with $N = 10^8$ and with locks is approximately 0.4816 seconds

3.2.1.2 Experiment Observations

According to the results first of all we observe the difference in computation cost between the functions where using $f(x) = x^2 \, with \, N = 10^8$ is much lower than the other cases of the same N. Actually the cost is more comparable with the function $f(x) = cos(x^2) \, with \, N = 10^3$. The most important observation is the time reduction as more threads are being used. In all cases with $N = 10^8$ when the number of threads that are being used is 8, time of execution is at its lowest. On the other hand at the case of $N = 10^3$, using 1 thread is the faster solution. This is due to the fact that the number of computations needed is very small. Thus, using only 1 thread is faster because the time saved from changing between threads makes a big impact when tasks are not many. These conclusions can be clearly observed on the graph below.



It is interesting to see the values printed on each thread. For example with 8 threads used we can see the progressive t update of the integral value during execution:

```
Current integral value: 0.597064
Current integral value: 1.19413
Current integral value: 1.22726
Current integral value: 1.2331
Current integral value: 1.26623
Current integral value: 1.25035
Current integral value: 1.23446
Current integral value: 1.24031
```

As the computation proceeds, the result gradually converges toward the final value of 1.24031. The small variances between updates are expected, since different threads contribute partial sums(local_sum) at different times.

3.2.2 Without Locks

To compute the integral without using mutexes, each thread calculates its own partial result (local_sum) for a specific portion of the interval, avoiding the use of a shared global variable. Instead of updating a common sum, every thread stores its result in a diffrent position of an array (table). Each thread determines the range of indices it is responsible for based on its id, with the total number of points N divided equally among the threads and the last thread handling any remaining elements. More specifically, each thread works on the range from

start_index = id * pieces to

end_index = (id + 1) * pieces (or end_index = N for the last thread).

```
int pieces = N / NUM_THREADS;

int start_index = id * pieces;

int end_index;
if (id == NUM_THREADS - 1) {
    end_index = N;
}
else {
    end_index = (id + 1) * pieces;
}
```

For every point in this range, the thread evaluates the function and updates its local sum using local_sum += 2 * f(x). After completing its computation, the thread stores its result in table[id]. Once all threads have finished execution, the main thread gathers all partial sums from the table array and computes their total. It then adds the values of f(a) and f(b) and multiplies the result by h / 2.0 to produce the final value of the integral. By storing each thread's result separately, the need for mutexes or locks is eliminated, avoiding race conditions while maintaining full parallelism.

3.2.2.1 Experiment Results

Results are calculated in seconds

Computing the integral of $f(x) = x^2 \; at \; [a: 0.0, \; b: 10.0]$
N = 100000000
Result= 333.333

Threads	first run	second run	third run	fourth run	fifth run	average
1	0.31713	0.298355	0.300115	0.300869	0.30126	0.303546
2	0.191368	0.157651	0.1511	0.159865	0.163264	0.16465
4	0.114156	0.100541	0.117731	0.102728	0.10482	0.107995
6	0.0832871	0.0839577	0.0808336	0.0862127	0.0856618	0.0839906
8	0.0733242	0.078072	0.0772248	0.0731461	0.0732597	0.0750054

Computing the integral of $f(x) = e^{-x^2} (Gaussian \; Function) \; at \; [a: -10.0, \; b: 10.0]$
N = 100000000
Result= 1.77245

Threads	first run	second run	third run	fourth run	fifth run	average
1	1.28473	1.31854	1.29113	1.30869	1.32895	1.30641
2	0.733473	0.798798	0.744934	0.684372	0.771314	0.746578
4	0.4585	0.412145	0.374589	0.425079	0.368122	0.407687
6	0.334621	0.321771	0.319187	0.305953	0.316725	0.319651
8	0.268183	0.280332	0.27346	0.287525	0.2635	0.2746

Computing the integral of $f(x) = cos(x^2) \; at \; [a: -50.0, \; b: 50.0]$
N = 100000000
Result= 1.24031

Threads	first run	second run	third run	fourth run	fifth run	average
1	1.60665	1.59536	1.60624	1.58717	1.58225	1.59553

2	0.912808	0.808668	0.861303	0.846654	0.836517	0.85319
4	0.470709	0.453395	0.454929	0.476896	0.448321	0.46085
6	0.39896	0.38949	0.3807	0.370198	0.366753	0.38122
8	0.337307	0.321617	0.335558	0.315105	0.330288	0.327975

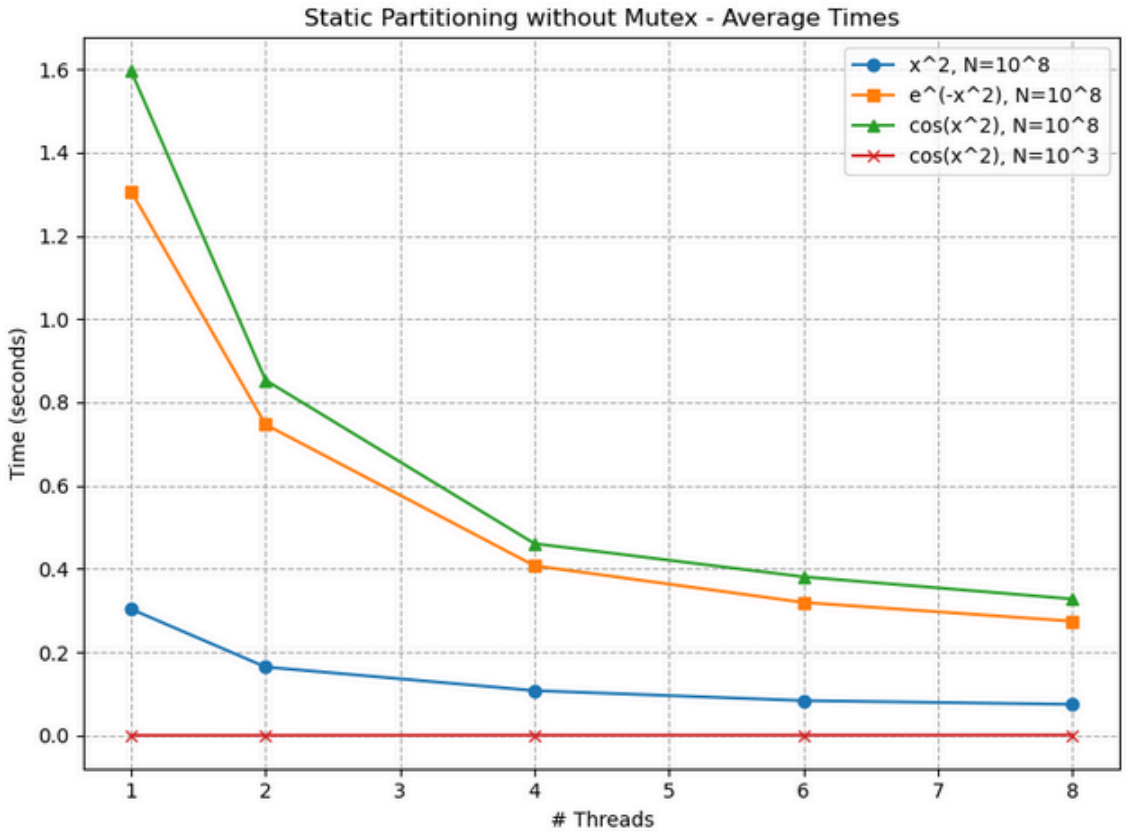
Computing the integral of $f(x) = cos(x^2)\ at\ [a: -50.0,\ b: 50.0]$
N = 1000
Result= 4.75051

Threads	first run	second run	third run	fourth run	fifth run	average
1	0.000788	0.0005566	0.0009211	0.0008342	0.0005087	0.0007217
2	0.000924	0.000575	0.0007323	0.0007011	0.0005822	0.0007029
4	0.0011691	0.0009173	0.0007976	0.0013081	0.0008955	0.0010175
6	0.0010845	0.0011698	0.0010477	0.0011901	0.0009686	0.0010921
8	0.0012621	0.0011331	0.0010956	0.0014853	0.0016466	0.0013245

Static Partitioning Average Experiment Time with $N = 10^8$ and with no locks is approximately **0.4939** seconds

3.2.2.2 Experiment Observations

According to the results without mutexes, the observations are very similar to those with mutexes. The computation time is slightly higher overall because the table used for storing each thread's partial results is heavier than using locks, which introduces additional memory overhead. As before, using $f(x) = x^2\ with\ N = 10^8$ is much faster compared to the other functions with the same N, and its cost is comparable to $cos(x^2)\ with\ N = 10^3$. The most important observation remains the reduction in execution time as more threads are used. For all functions with $N = 10^8$, the execution time is at its lowest when 8 threads are used. On the other hand, in the case of $N = 10^3$, using a single thread is faster because the number of computations is very small, and the overhead of switching between threads would "win" against the benefit of parallelism. These trends can be clearly seen in the corresponding graph below.



As with using locks, in this implementation we can see on every step the progressive update of the integral from the total sum of the elements of the array that is being used.

```
Current integral value: 0.00584501
Current integral value: -0.010038
Current integral value: 0.0230901
Current integral value: 0.620154
Current integral value: 1.21722
Current integral value: 1.25035
Current integral value: 1.23446
Current integral value: 1.24031
Integral = 1.24031
```

T Here the values show larger variances in the beginning because the partial sums from the threads are combined at different times, without strict synchronization. But as before, even now as more thread results are aggregated, the integral quickly stabilizes and converges to the final value of 1.24031.

4 Cyclic Scheduling

Cyclic Scheduling is a workload distribution method where the iterations are shared among the threads in a round-robin way instead of giving each thread one continuous block. Each thread starts from its own ID and processes iterations with a

fixed step equal to the number of threads, covering the whole range gradually. This helps achieve better load balancing, especially when some iterations take more time than others. Each thread calculates a local sum independently and at the end adds it to the shared result. On the other hand in static partitioning, synchronization mechanisms are needed to safely update the shared variable, but the overhead is smaller since this update usually happens only once per thread.

4.1 The Application

In this implementation, the integral is computed using cyclic scheduling, where the iterations are shared among the threads in a round-robin way. Each thread starts from its own id and processes elements with a fixed step equal to the number of threads for(i = id; i < N; i += NUM_THREADS){...}, so all threads work on different parts of the interval at the same time. For every iteration, each thread calculates the value of the function and stores the result in a local variable (local_sum): local_sum += 2 * f(x);

```
for (int i = id; i < N; i = i + NUM_THREADS) {
    double x = a + i * h;
    local_sum += 2 * f(x);
}
```

This allows the threads to work independently without interfering with each other during the computation. After finishing its work, each thread adds its local result to the shared variable sum. Because sum is shared, a mutex is used to avoid errors. Firstly the thread locks the mutex, updates sum, and then unlocks it. When all threads are done, the main thread adds f(a) and f(b) and multiplies the result by h / 2.0 to compute the final integral. This method shares the work more evenly between the threads and keeps the synchronization cost low, because each thread updates the shared variable only once.

4.2.1 Experiment Results

Results are calculated in seconds

Computing the integral of $f(x) = x^2$ *at* $[a: 0.0, b: 10.0]$

N = 100000000

Result= 333.333

Threads	first run	second run	third run	fourth run	fifth run	average
1	0.292449	0.290247	0.288265	0.287581	0.291967	0.290102
2	0.16372	0.144742	0.150535	0.14571	0.153453	0.151632
4	0.102207	0.115382	0.107737	0.104057	0.105855	0.107048
6	0.0756777	0.076399	0.0803299	0.0799284	0.0840954	0.0792861
8	0.074756	0.0739766	0.0726691	0.0781011	0.0809797	0.0760965

Computing the integral of $f(x) = e^{-x^2} (Gaussian\ Function)\ at\ [a: -10.0,\ b: 10.0]$
N = 100000000
Result= 1.77245

Threads	first run	second run	third run	fourth run	fifth run	average
1	1.35799	1.31491	1.317	1.30292	1.31632	1.32183
2	0.677077	0.645975	0.672493	0.693285	0.653704	0.668507
4	0.432781	0.430298	0.426428	0.355943	0.450246	0.419139
6	0.320301	0.328233	0.335528	0.327907	0.32986	0.328366
8	0.294301	0.277375	0.283285	0.290211	0.282533	0.285541

Computing the integral of $f(x) = cos(x^2)\ at\ [a: -50.0,\ b: 50.0]$
N = 100000000
Result= 1.24031

Threads	first run	second run	third run	fourth run	fifth run	average
1	1.69932	2.19494	1.61758	1.73193	1.64155	1.77706
2	0.945424	0.814933	0.853277	0.918466	0.879856	0.882391
4	0.601484	0.509288	0.480326	0.492016	0.498674	0.516358
6	0.384696	0.369533	0.369339	0.374996	0.365501	0.372813
8	0.329688	0.322458	0.320605	0.336137	0.325741	0.326926

Computing the integral of $f(x) = cos(x^2)\ at\ [a: -50.0,\ b: 50.0]$

N = 1000
Result= 4.75051

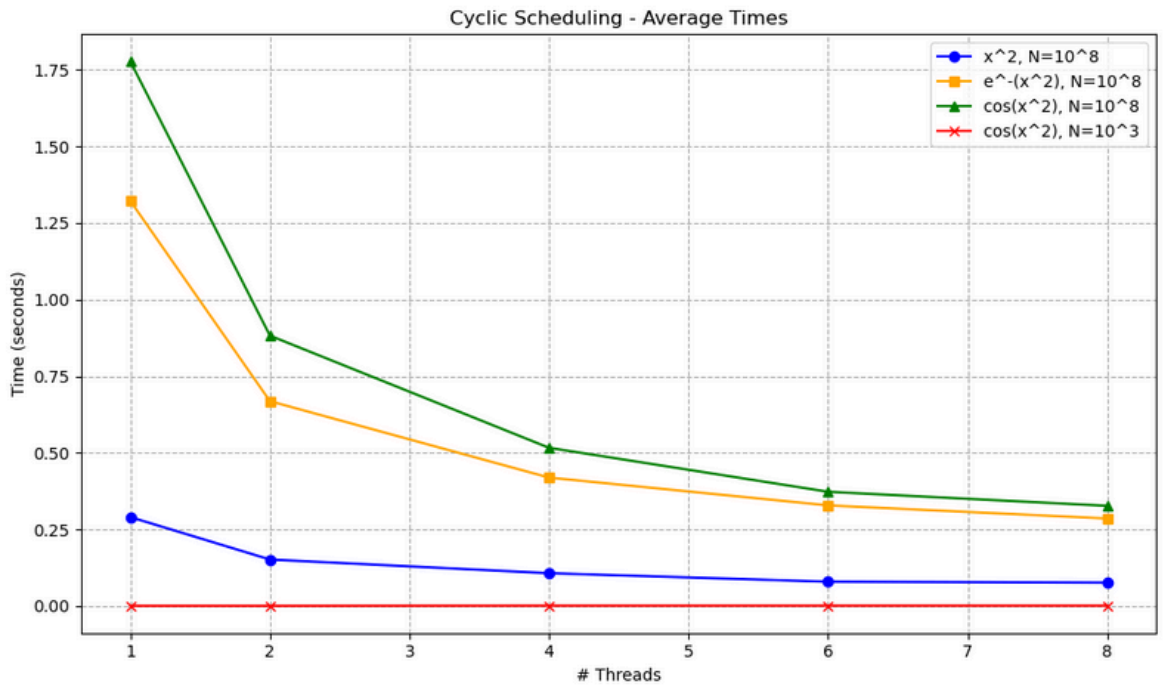
Threads	first run	second run	third run	fourth run	fifth run	average
1	0.0015343	0.000513	0.0005641	0.0005128	0.0005066	0.0007261
2	0.00062	0.00061	0.0006902	0.0005692	0.0006066	0.0006192
4	0.0008679	0.0016555	0.0014572	0.0009247	0.0008088	0.0011428
6	0.0013206	0.0008861	0.001193	0.0011336	0.0012452	0.0011557
8	0.0015084	0.0011211	0.0010226	0.0011755	0.0010904	0.0011836

Cyclic Scheduling Average Experiment Time with $N = 10^8$ is 0.507006

4.2.2 Experiment Observations

According to the results first of all we observe the difference in computation cost between the functions where using $f(x) = x^2$ with N = 10^8 is much lower than the other cases of the same N. The functions $f(x) = e^{-x^2}$ and $f(x) = cos(x^2)$ require more time due to their higher computational complexity. The most important and interesting observation is again the time reduction as more threads are being used. In all cases where we use N = 10^8, the execution time decreases significantly as the number of threads goes up, reaching its lowest values when all 8 threads are being used. This shows that cyclic scheduling distributes the workload efficiently among threads, keeping them all active during execution.

On the other hand, in the case of N = 10^3 for the function $cos(x^2)$, using only 2 threads gives the best performance, while increasing the number of threads further leads to worse execution times. This happens because the workload is way smaller than the other cases and the overhead of thread management becomes more significant than the computation itself. Thus, using too many threads in small tasks reduces performance. These conclusions can be clearly observed on the graph below.



5 Dynamic Scheduling

Dynamic scheduling is a workload distribution method where iterations are not assigned to threads beforehand but are given dynamically during the execution of the algorithm. Each thread takes some work, and when it finishes, it requests more, so faster threads end up doing more iterations while slower ones do less. This helps achieve better load balancing, especially when iterations do not all take the same time. However, it is important to note that synchronization mechanisms need to be used in order to safely access the shared pool of remaining work as well as the shared variables. In this application locks implemented by mutexes will be used.

5.1 The Application

In this implementation, the integral is computed using dynamic scheduling. The iterations are divided into small pieces of work of size K, and threads take pieces one by one. Each thread locks the tmutex mutex to get the next piece (t = taskid++) and then unlocks it.

```
pthread_mutex_lock(&tmutex);
t = taskid++;
pthread_mutex_unlock(&tmutex);
if (t >= NTASK) break;
```

For each piece of work, the thread goes through all its iterations using the loop for (int i = t * K; i < (t+1)*K; i++), which calculates f(x) for every step in that piece and stores the result in a local variable local_sum += 2 * f(x);. With this way, threads can work on different pieces of the integral without interfering with each other. After finishing a piece of work, each thread adds its local_sum to the shared variable sum. A mutex is used when updating sum to avoid errors.

```
for (int i = t * K ; i < (t+1)*K; i++) {
    double x = a + i * h;
    local_sum += 2 * f(x);
}

pthread_mutex_lock(&mutex);
sum += local_sum;
pthread_mutex_unlock(&mutex);
```

This method balances the work automatically because faster threads take more chunks and slower threads take fewer, while keeping the synchronization simple by updating sum only once per work-piece.

5.2.1 Experiment Results

Results are calculated in seconds

Computing the integral of $f(x) = x^2$ *at* $[a: 0.0, b: 10.0]$

N = 100000000

K = 100

Result= 333.333

Threads	first run	second run	third run	fourth run	fifth run	average
1	0.351971	0.335445	0.341634	0.33366	0.335427	0.339627
2	0.275747	0.318734	0.305187	0.291218	0.297101	0.297597
4	0.286709	0.290849	0.27791	0.273295	0.271759	0.280104
6	0.288765	0.283022	0.276816	0.285828	0.285133	0.283913
8	0.278513	0.276845	0.282294	0.277813	0.28081	0.279255

Computing the integral of $f(x) = e^{-x^2} (Gaussian\ Function)\ at\ [a: -10.0,\ b: 10.0]$

N = 100000000

K = 100

Result= 1.77245

Threads	first run	second run	third run	fourth run	fifth run	average
1	1.39339	1.32847	1.32566	1.38857	1.35809	1.35884
2	0.89465	0.7696	0.781794	0.76745	0.790855	0.80087
4	0.5793	0.483884	0.493609	0.469545	0.478751	0.501018
6	0.457373	0.434833	0.429655	0.429845	0.482106	0.446762
8	0.399203	0.39761	0.39323	0.385745	0.394919	0.394141

Computing the integral of $f(x) = cos(x^2)\ at\ [a: -50.0,\ b: 50.0]$

N = 100000000

K = 100

Result= 1.24031

Threads	first run	second run	third run	fourth run	fifth run	average
1	1.85473	1.62394	1.63522	1.71534	1.64178	1.6942
2	1.00121	0.919885	0.912251	0.938568	0.93827	0.942037
4	0.56585	0.5471	0.557172	0.538098	0.541335	0.549911
6	0.516315	0.479797	0.477387	0.49452	0.471557	0.487915
8	0.429501	0.508955	0.422396	0.445275	0.415676	0.444361

Computing the integral of $f(x) = cos(x^2)\ at\ [a: -50.0,\ b: 50.0]$

N = 1000

K = 100

Result= 4.75051

Threads	first run	second run	third run	fourth run	fifth run	average
1	0.0005801	0.0006148	0.0005105	0.0008613	0.0005443	0.0006222

2	0.0008742	0.0006975	0.0006663	0.0007288	0.0010524	0.0008038
4	0.0007233	0.000567	0.0005351	0.0009657	0.0006943	0.0006970
6	0.000906	0.0019357	0.0010006	0.0008081	0.0014153	0.0012131
8	0.0012636	0.0015279	0.0011542	0.0010977	0.001591	0.0013268

Computing the integral of $f(x) = cos(x^2)\ at\ [a: -50.0,\ b: 50.0]$

N = 100000000

K = 10000

Result= 1.24031

Threads	first run	second run	third run	fourth run	fifth run	average
1	1.66308	1.61922	1.61042	1.61819	1.61953	1.62609
2	0.842608	0.840586	0.849216	0.833056	0.856672	0.844428
4	0.49657	0.470862	0.448376	0.462534	0.478586	0.471386
6	0.43181	0.380868	0.360585	0.36355	0.361741	0.379711
8	0.334032	0.312401	0.313882	0.311777	0.323178	0.319054

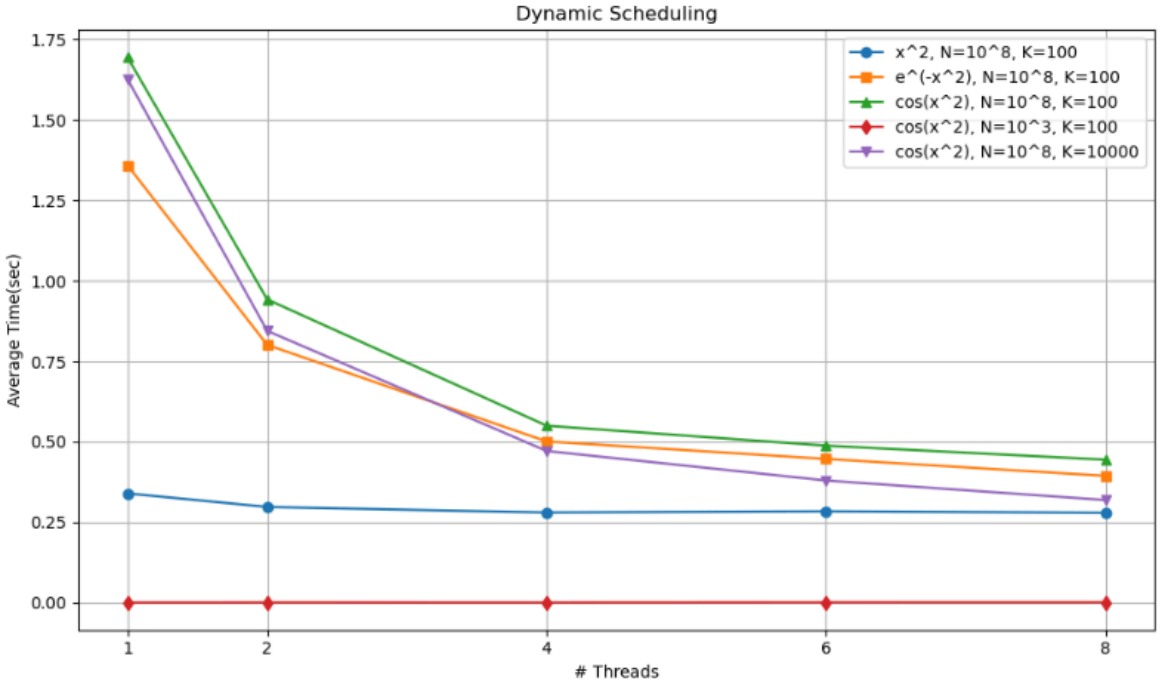
Dynamic Scheduling Average Experiment Time with $N = 10^8\ and\ K = 100$ is 0.60697 seconds

5.2.2 Experiment Observations

According to the results of the experiment, as expected there is similar behaviour with the cyclic scheduling when we see and compare the results from the functions with $N = 10^8$ to the one with $N = 10^3$. At the case of $N = 10^8$ the computation becomes much faster by increasing the number of threads involved. As analyzed before, the workload is very heavy and as a result parallelism contributes a lot. This explains why the faster time occurs when 8 threads are used. On the other side when we look at the results of $cos(x^2)$ with $N = 10^3$ we observe that parallelism does not contribute as execution time is at its lowest when we use only 1 thread.

Another interesting observation is that by increasing K from 100 to 10000, all execution times (with the same N) have been improved. This is normal and totally expectable because each piece of work is bigger, so threads spend more time computing and less time locking and unlocking the mutex. At the same time, the

number of pieces is smaller, allowing the program to run faster overall. All of these trends can be observed in the following graph.



6 General Observations

Taking all the experiments into consideration, as the workload increases, parallelism becomes more noticeable. In all the parallel methods, when $N = 10^8$ the execution time is lower as the number of threads is increasing. On the other side when $N = 10^3$ computation time is less when we use only 1 thread. This happens because tasks and computations are not many. Moreover we observe that time spent in locking and unlocking mutexes has a much bigger effect than mathematical operations. Thus, using a lot of threads to achieve parallelism when the number of computations is low, is not great for effectiveness.

Another interesting fact that is noticeable is that because the computer used for the experiments has 4 cores, the time reduction from 1→2 threads and 2→4 threads is larger than when more than 4 threads are used. This is expected, as true parallelism is fully achieved when the number of threads matches the number of physical cores. When using more than 4 threads, the additional threads share the same cores, leading to context switching and scheduling overhead, which reduces the performance gain per extra thread.